

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	7
1 ОБЗОР ЛИТЕРАТУРЫ.....	9
1.1 Классические методы PID – ZMP	10
1.2 Модель-предиктивное управление (Model Predictive/MPC)....	10
1.3 Обучение с подкреплением (Reinforcement Learning/RL)	11
2 ТЕОРЕТИЧЕСКОЕ ОПИСАНИЕ.....	14
2.1 Обзор моделей.....	14
2.2 Планирование движения.....	18
2.3 Алгоритмы управления.....	21
3 ПОСТАНОВКА ЗАДАЧИ.....	25
4 СИМУЛЯЦИЯ И РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ.....	27
4.1 Симуляция	29
4.2 Анализ результатов экспериментов.....	37
ЗАКЛЮЧЕНИЕ	38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39

ВВЕДЕНИЕ

В последние несколько десятилетий робототехника стала стремительно развиваться в направлении создания и разработки автономных и полуавтономных мобильных систем, которые способны функционировать в условиях неидеальной изменяющейся среды, а именно при наличии неровных поверхностей и воздействия внешних сил. Поэтому вскоре встал вопрос, какой же подвид роботов по типу механической части больше подходит для данной задачи.

Общеизвестен тот факт, что мобильные роботы классифицируются по среде эксплуатации: наземные, воздушные и водные; Наземные роботы делятся на колесных, гусеничных, ползающих и шагающих роботов. В связи с большей площадью соприкосновения с поверхностью колесные и гусеничные роботы больше используются для передвижения по относительно неровным поверхностям, как пример можно взять луноходы или марсоходы, шагающие же роботы стараются имитировать движения животных или человека в зависимости от конфигурации, поэтому они стали более предпочтительными для преодоления комплексных препятствий в виде лестниц и завалов. Как можно заметить в видео, робот Atlas от Boston Dynamics, демонстрирует высокий уровень адаптивности за счет своей конструкции [1].

Такой вид роботов принято называть гуманоидным, так как он своим внешним видом и походкой подobaет человеку. Большое количество степеней свободы, изначально неустойчивое состояние системы, нелинейная динамика, а также малое количество точек опоры делает задачу управления таким роботом комплексной и высокoзатратной. Шестиногие и восьминогие роботы же обеспечивают высокую устойчивость в силу большого количества точек опоры, однако чаще всего они являются слишком энергозатратными. Поэтому на данный момент самым актуальным вариантом является механизм на четырех ногах, например робот-собака Spot от Boston Dynamics. Разработка подобной системы требует глубокого понимания алгоритмов управления и планирования движением, так как динамика системы включает в себя постоянный анализ положения центра масс, распределение сил при опоре, что критично для поддержания устойчивости.

Цель данной работы – исследование существующих алгоритмов планирования и управления движением шагающих роботов, таких как MPC, WBC, Inverse Dynamics Control, DRL, а также изучение классических подходов с заранее известными траекториями походок и использованием ПД регулятора. Применение некоторых из этих алгоритмов к виртуальной модели квадрупеда Unitree A1, используя физический движок для моделирования робототехнических систем Mujoco и проведение сравнительного анализа их эффективности при походках trot по метрикам устойчивости (отклонения центра масс от заданной траектории ZMP), энергоэффективности и быстродействия. Тестирование будет проводиться в одинаковых условиях ровной поверхности для простоты исследования.

1 ОБЗОР ЛИТЕРАТУРЫ

Чаще всего модель простого колесного робота можно представить в виде материальной точки и все передвижения отобразить в двумерных координатах. Однако, если мы будем говорить о шагающих роботах, то количество степеней свободы возрастет, а также в связи с изначальной нестабильностью системы представление робота будет несколько другим. Всего в классических шагающих роботах 2 – 3 степени свободы в каждой ноге.

Для описания таких систем требуются более сложные модели, которые учитывают как динамику, так и взаимодействие с опорной поверхностью. Поэтому в зависимости от того какую задачу мы рассматриваем, это может быть управление балансом, планирование траекторий или та же симуляция, применяются соответствующие им модели.

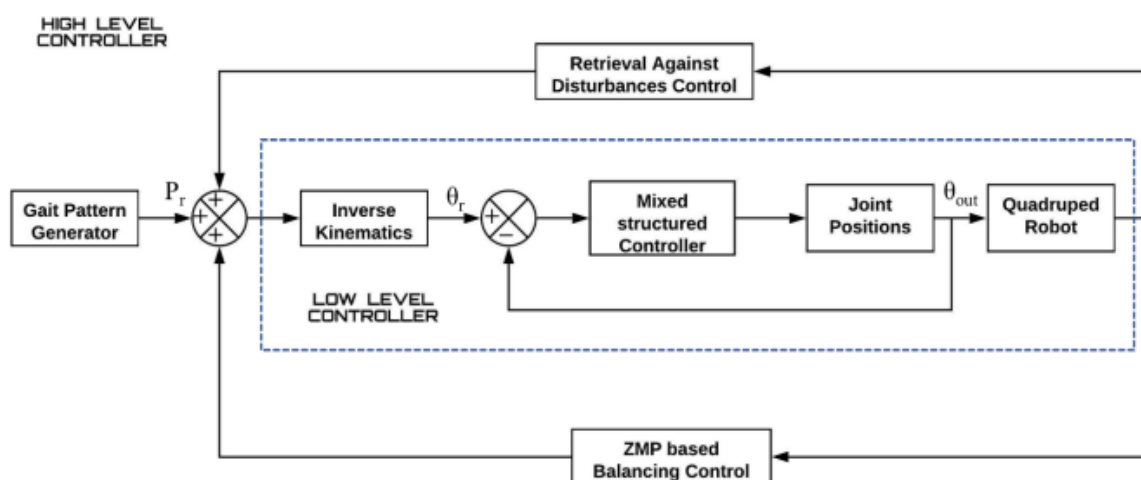


Рисунок 1 – Архитектура управления квадрупедом[22]

В общем случае управление шагающим роботом можно разбить на две составляющие, низкоуровневую и высокоуровневую. Высокоуровневый контроллер включает в себя генератор походки. Походка предоставляет нам траекторию движения робота, например по прямой.

Низкоуровневый контроллер же занимается тем, что он генерирует углы сочленений для каждой ноги используя задачу обратной кинематики. Например, может использоваться, как и ПИД регулятор, так и управление H_2/H_∞ . Стоит отметить, что данные методы управления работают хорошо только в заранее известных условиях и не обладают способностью быстро и динамично адаптироваться к новым условиям, поэтому чтобы обойти эти ограничения изучим методы ниже.

1.1 Классические методы PID – ZMP

Управление устойчивостью же происходит на основе ZMP – технологии (Zero Moment Point, «точка нулевого момента»). В некоторых случаях ZMP можно рассматривать как центр давления на стопу. Это точка, относительно которой все моменты сил уравниваются. Поддержание ZMP в пределах площади опоры (например, стопы) необходимо для предотвращения опрокидывания робота. Эта концепция используется для анализа, синтеза и управления двуногой походкой роботов. ZMP — это точка на опорной поверхности, относительно которой суммарный момент инерционных сил и сил тяжести не имеет компонентов вдоль горизонтальных осей. Важно отметить, что ZMP и центр давления (CoP) — это разные понятия, хотя в динамически сбалансированной походке они совпадают. CoP — это точка, в которой можно заменить давление между стопой и землей силой.

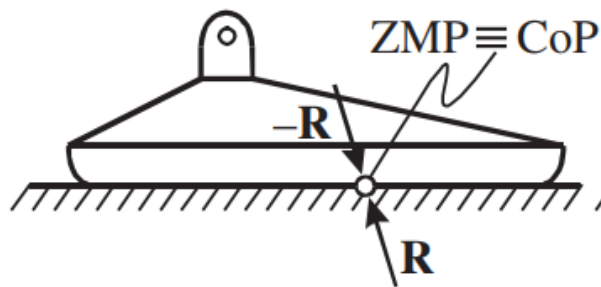


Рисунок 2 – Пример расположения ZMP[21]

1.2 Модель-предиктивное управление (Model Predictive/MPC)

Модельно-предиктивное управление (MPC) — это метод управления, в котором используется математическая модель системы для прогнозирования её поведения на ограниченном временном интервале, называемом горизонтом предсказания. На основе текущего состояния системы и прогнозируемых будущих значений вычисляются оптимальные управляющие воздействия, соответствующие заданной целевой функции и учитывающие ограничения системы. Затем через определённый временной шаг процесс измерения, прогнозирования и оптимизации повторяется с обновлённым горизонтом.

Несмотря на эффективность классического MPC, в реальных системах часто приходится учитывать нелинейную динамику, что требует применения более сложных методов. В таких случаях используется нелинейный MPC (NMPC), рекурсивно решающих задачи оптимального управления с конечным горизонтом, используя нелинейные модели системы. Существуют и другие вариации данного метода.

Вариации MPC:

- NMPC (Nonlinear Model Predictive Control) или нелинейное модельное прогнозное управление – это продвинутая стратегия управления, которая используется для управления роботами и другими сложными системами. Она сочетает в себе преимущества модельного прогнозирующего управления (MPC) с возможностью работы с нелинейными моделям. Пример: управление гуманоидным роботом Atlas от Boston Dynamics, где NMPC применяется для балансировки на неровных поверхностях.
- Convex Model Predictive Control (Convex MPC) — это метод управления, который использует выпуклые оптимизационные задачи для прогнозирования и управления движением роботов. Он в особенности полезен для систем с высоким количеством степеней свободы, таких как шагающие роботы, где необходимо учитывать сложные динамические ограничения и обеспечивать устойчивость. Он формулируется как задача квадратичного программирования, где целевая функция минимизирует отклонение от заданной траектории, а ограничения включают динамику системы, ограничения на управление и состояние. Convex MPC использовался для управления MIT Cheetah 3, четвероногим роботом и в последствии он смог развивать скорость до 3 м/с.
- Гибридная конфигурация Whole Body Control и MPC. В работе [4] рассматривается именно такой подход, где MPC используется для вычисления оптимальных профилей сил реакции (реакции опоры) на длительном горизонте прогнозирования с использованием упрощённой модели, а WBC (в данном случае Whole-Body Impulse Control, WBIC) вычисляет команды для управления суставами (моменты, положения и скорости) на основе сил реакции, вычисленных MPC. В отличие от традиционных WBC, которые пытаются отслеживать заданные траектории тела, новый контроллер фокусируется на отслеживании команд сил реакции, что позволяет достичь высокой скорости.

1.3 Обучение с подкреплением (Reinforcement Learning/RL)

Применение машинного обучения, в частности обучения с подкреплением (RL), активно развивается в области управления и планировании траекторий роботов. Применяя обучение с подкреплением, робот может автономно обучиться оптимальным стратегиям передвижения через прямое взаимодействие с окружающей средой, без необходимости

моделирования динамики или явного задания траекторий. Это делает такой способ подходящим для решения задач движения шагающих роботов.

Существуют два основных подхода к обучению робота передвижению:

- Обучение в реальном мире (Real life RL)

Этот вариант имеет место быть, но он не является оптимальным, так как робот никак не застрахован от физических повреждений при падениях. Он работает в режиме реального времени и это ограничивает объем данных для обучения из-за низкой скорости сбора данных. Также реальные роботы не могут работать постоянно, постоянное обучение ходьбе требует много энергии, поэтому они имеют тенденцию разряжаться.

- Обучение в симуляции (Sim-to-Real)

Вся суть заключается в том, что робот изначально обучается в виртуальной среде, проводится большое количество попыток, дабы он собрал данные. Плюсом является тот факт, что в виртуальной среде можно поиграть со сценариями и создавать редкие/опасные случаи не рискуя поломкой робота, в случае если он упадет. Недостатком же является «reality gap» Сильвиан Кус в своей работе [6] предполагает, что «...разрыв в основном возникает из-за конфликта между эффективностью решений в симуляции и их переносимостью из симуляции в реальность: лучшие решения в симуляции часто опираются на некорректно смоделированные явления (например, наиболее динамичные)».

В исследовании [2] показали применение метода глубокого RL который позволяет переносить обученные в симуляции управляющие роботом политики, на реальную систему(sim-to-real). Весь обучающий процесс проходит в гибридной симуляционной среде, где есть физическая достоверная модель и там же проводится нейросетевое моделирование динамики приводов. Нужда в четком знании параметров динамики робота отпадает, поэтому можно считать, такой метод эффективным, особенно в условиях неизвестной или меняющейся среды.

В статье [5], авторы пытаются решить вопрос создания более реалистичной анимации физических персонажей с помощью объединения подхода motion capture и глубокого обучения с подкреплением. Они используют алгоритм Proximal Policy Optimization, обучающий агентов действовать в среде и непосредственно оптимизирующий политику. Для экспериментов были использованы модели гуманоида, Т-рекса, дракона и робота Atlas.

Таблица 1 – Сравнение алгоритмов по критериям

Критерий	PID	MPC	RL
Точность	Средняя(очень зависит от настройки коэффициентов)	Высокая (оптимизация на горизонте)	Высокая (после обучения)
Устойчивость	Только в стабильных условиях	Хорошая (учет будущих состояний системы)	Адаптивная
Скорость работы	Быстрая (нет сложных вычислений)	Средняя/высокая (в зависимости от модели)	Быстрая
Гибкость	Низкая (жестко заданная логика)	Средняя (нужна точная модель)	Высокая (самообучение)
Сложность	Низкая	Высокая	Очень высокая

Основываясь на этом проведенном обзоре, было решено произвести сравнительный анализ базовых алгоритмов: PID-ZMP и MPC.

2 ТЕОРЕТИЧЕСКОЕ ОПИСАНИЕ

2.1 Обзор моделей

Управление движением четвероногого робота является непростой задачей, так как степеней свободы много, взаимодействие чаще всего происходит с неоднородной средой, да и сложность у динамической модели высокая. Поэтому, чтобы не загружать процессор трудоемкими вычислениями для синтеза алгоритмов управления, принято использовать упрощенные модели, которые позволяют сохранять основные физические свойства, но исключают второстепенные детали. Такой подход особенно удобен, когда сами алгоритмы являются требовательными, например Deep Reinforcement Learning.

Модель точечной массы

Во всех школьных физических задачах, делалось одно главное допущение: «Представим, что тот или иной объект, это материальная точка». Такое упрощение предполагает, что размерами самого тела можно пренебречь и вся масса сконцентрирована в одной точке.

Этот подход очень часто используется для моделирования передвижения шагающих роботов в том числе. Стэфан Карон в своей работе [18] выделил два допущения: достаточность актуации и сосредоточенность массы. Первый термин подразумевает, что робот может быть упрощен до точечной массы, только если его приводы могут создавать достаточно вращающего момента для управления сочленениями ног. Тогда движение звеньев можно аппроксимировать и сконцентрировать наше внимание на движении «плавающей базы тела».

Также предполагается, что вся масса шагающего робота находится в его центре масс, то есть квадрупед превращается в точку в пространстве с массой m , и мы наблюдаем как эта точка передвигается под воздействием результирующих контактных сил направленных через центр масс.

С упрощенной динамикой, выходят следующие уравнения модели, описывающие поступательное и вращательное движение:

$$m\ddot{p}_G = f + mg, \quad (1)$$

$$\dot{L}_G = \tau_G = 0, \quad (2)$$

где $\ddot{p}_G$ – ускорение центра масс (ЦМ), f – результирующая сила контакта с опорной поверхностью, m – масса робота, L_G – производная углового момента относительно ЦМ, τ_G – момент силы контакта относительно ЦМ.

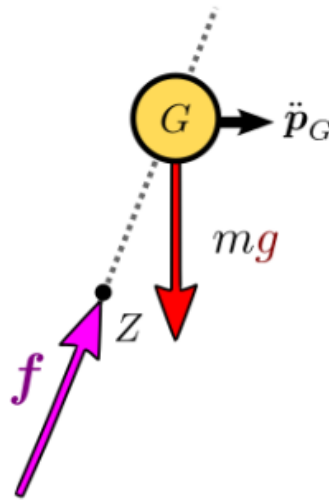


Рисунок 3 – Модель точечной массы [18]

Эта модель лежит в основе таких упрощенных моделей как линейный обратный маятник (LIPM) и пружинный обратный маятник (SLIP).

Модель линейного обратного маятника (Linear Inverted Pendulum Model)

Когда мы думаем о гуманоидных или просто двуногих роботах линейная модель перевернутого маятника – это та модель, которую используют для синтеза управляющего балансом регулятора. В момент, когда мы переносим весь вес на одну ногу и другая нога висит в воздухе, мы – люди представляем из себя неустойчивую систему обратного маятника.



Рисунок 4 – Человек-маятник

Здесь мы вспоминаем о модели точечной массы, представляя весь робот как материальную точку, которую мы закрепляем на невесомой ноге, такой подход позволяет отслеживать движение центра масс в 2-мерном и 3-хмерном пространстве.

Мы хотим, чтобы наша модель оставалась линейной поэтому угловой момент у центра масс отсутствует ($L_G = 0$), а также, высота на которой находится центр масс относительно опорной поверхности должна оставаться неизменной.

Каджита в своей работе 2001 г. [19] рассматривал 3-хмерную модель линейного обратного маятника для передвижения двуногого робота.

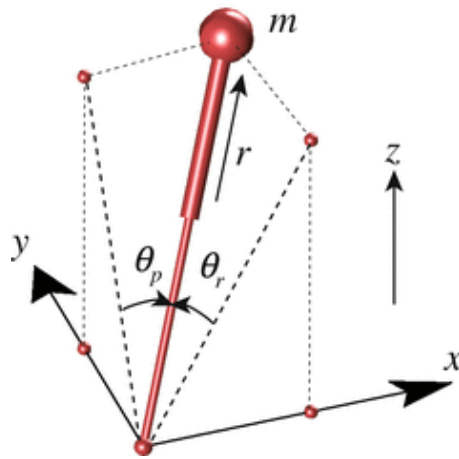


Рисунок 5 – Модель 3-хмерного маятника [21]

Уравнения движения 3-хмерного перевернутого маятника:

$$m \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = (J^T)^{-1} \begin{bmatrix} \tau_r \\ \tau_p \\ f \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix}. \quad (3)$$

где J – матрица Якоби, связывающая углы θ_r, θ_p и длину ноги r с координатами (x, y, z) , а (τ_r, τ_p, f) моменты актуатора и сила соответствующие (θ_r, θ_p, r) .

$$S_r \equiv \sin \theta_r, S_p \equiv \sin \theta_p, D \equiv \sqrt{1 - S_r^2 - S_p^2},$$

$$J = \frac{\partial p}{\partial q} = \begin{pmatrix} 0 & rC_p & S_p \\ -rC_r & 0 & -S_r \\ -rC_rS_r/D & -rC_pS_p/D & D \end{pmatrix}, C_r \equiv \cos \theta_r, C_p \equiv \cos \theta_p.$$

Такая модель помогает планировать траектории центра масс во время передвижения и снижать вычислительную сложность, так как модель нелинейная. Но в этом подходе делается одно очень нереалистичное для походки людей и животных допущение – условие о постоянной высоте центра масс и пренебрежение амортизацией. Прыжки, бег или другие высокоактивные передвижения будут выполнены с меньшей точностью, поэтому перейдем к обзору следующей модели.

Модель перевернутого пружинного маятника (Spring-Loaded Inverted Pendulum)

В основе этой модели все еще лежит принцип точечной массы, но теперь эта масса прикреплена к безмассовой ноге-пружине.

Параметры модели:

- m – точечная масса;
- k – жесткость пружины;
- l_0 – длина ноги в несжатом состоянии;
- θ – угол отклонения ноги;
- r – текущая длина ноги, может меняться со временем.

Само движение можно разделить на две основные фазы: фаза полета и фаза опоры, наглядно это можно увидеть на рисунке ниже.

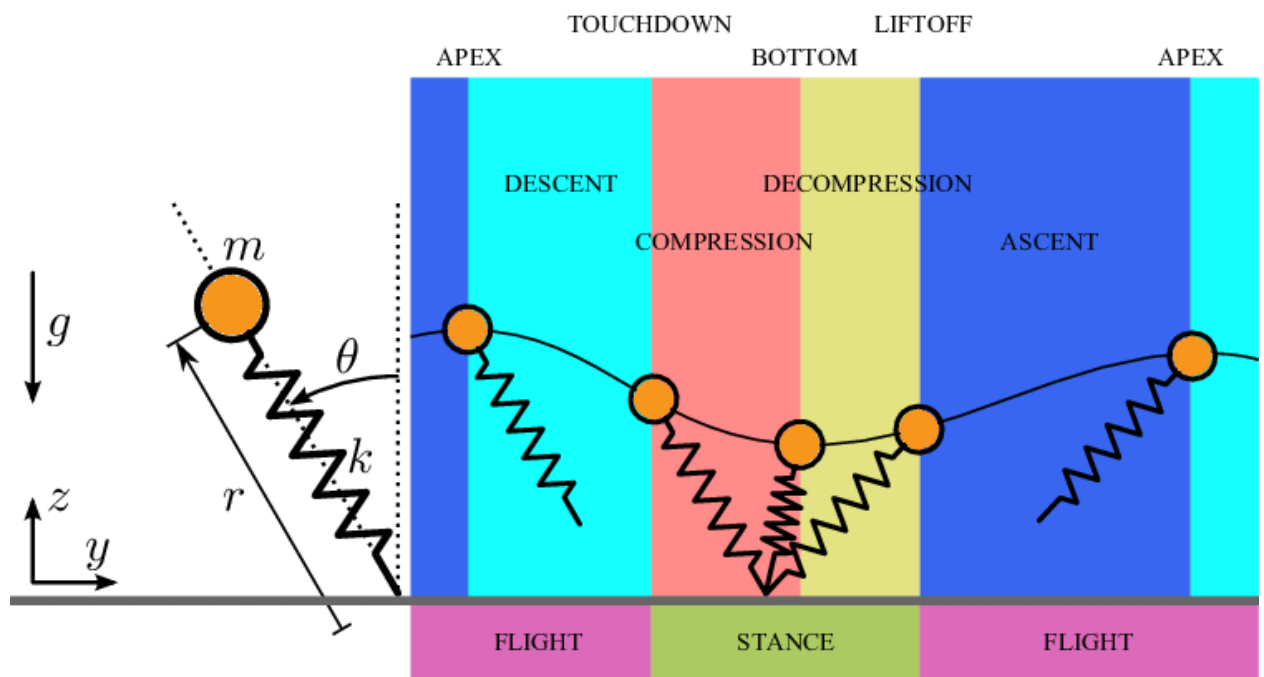


Рисунок 6 – Фазы SLIP – модели [24]

При фазе “полета” модель свободно падает под действием силы тяжести и траектория напоминает параболу, а во время фазы опоры нога находится в контакте с поверхностью, сжимается ($r < l_0$) и потом растягивается ($r > l_0$), вновь отправляясь в фазу полета.

Суммарная энергия создается из кинетической энергии самой массы и потенциальной, которую мы приобретаем во время контакта с землей.

$$T = \frac{m}{2}(\dot{r}^2 + r^2\dot{\theta}^2), \quad (4)$$

$$U = \cos \theta + \frac{k}{2}(l_0 - r)^2. \quad (5)$$

Применим уравнение Эйлера-Лагранжа и получим следующую систему дифференциальных уравнений 2-ого порядка, описывающие радиальное и угловое движение:

$$m\ddot{r} - mr\dot{\theta}^2 + \cos \theta - k(l_0 - r) = 0, \quad (6)$$

$$mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} - \sin \theta = 0. \quad (7)$$

Уравнение (6) описывает движение вдоль ноги, где первое слагаемое $m\ddot{r}$ – радиальное ускорение, $mr\dot{\theta}^2$ – центробежная сила, $\cos \theta$ – проекция силы тяжести и $k(l_0 - r)$ – сила упругости. Уравнение (7) соответствует изменению углового момента, $mr^2\ddot{\theta}$ – момент инерции, $2mr\dot{r}\dot{\theta}$ – кориолисова сила, $\sin \theta$ – вращательный момент.

Несмотря на свою реалистичности SLIP-модель имеет свой ряд ограничений, безмассовость ноги, в реальной жизни такие массивные элементы как ноги имеют инерцию и соответственно влияют на динамику движения. Не учитывается наличие приводов в ногах, то есть нога – это просто прямая пружинка.

2.2 Планирование движения

Основным и даже начальным этапом для разработки системы управления является планирование походки, нам надо знать когда, в какой

последовательности и сколько по времени происходит контакт конечностей с опорной поверхностью. Хорошее планирование траектории перемещения ноги предотвращает робота от падения.

Походки

Главная задача ученых на данный момент – это сделать перемещения роботов максимально приближенным к движениям животных. Поднятие ноги и приземление ее в том или ином порядке может считаться за походку, у четвероногих млекопитающих их большое количество. Различаются они в зависимости от скорости передвижения и поверхности по которой они ходят.

Выделяется два вида походок:

- Симметричные: Шаг, рысь и пэйсинг
- Асимметричные: Рысь с прыжком, скок и галоп

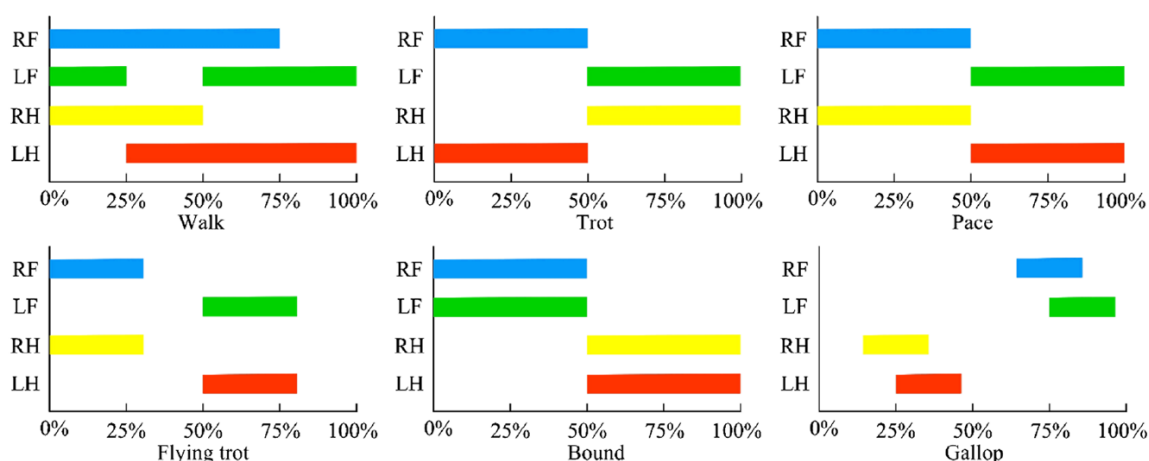


Рисунок 7 – Пример диаграмма контакта ног во время походок [20]

Конечный автомат (Finite State Machine/FSM)

Конечный автомат – это абстракция, которая позволяет проектировать алгоритмы, они работают как реагирующие системы. Примером может послужить светофор, у которого обычно три состояния: «Красный», «Желтый» и «Зеленый». Длительность у каждого состояния 30 сек., 5 сек., 25 сек. соответственно. Правила перехода триггерятся по истечению времени как описано на диаграмме ниже (см. Рисунок 10)

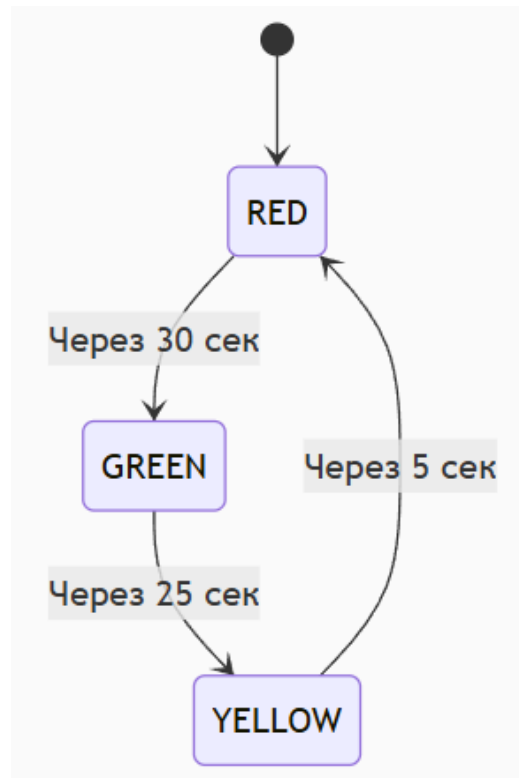


Рисунок 8 – Диаграмма конечного автомата для светофора

В случае шагающих роботов конечный автомат используется для координации ног по фазам опора(stance), мах (swing) в зависимости от заявленной логики походки.

Траектория движения ног

Когда нога переходит из фазы “stance” в фазу “swing” и обратно, у “стопы” происходит перемещение по какой-то оптимальной траектории, неудачное построение траектории может привести к потере равновесия, поэтому рассмотрим три основных метода ниже.

1. Полиномиальные траектории

Кубический сплайн (полином 3-ей степени) имеет следующее уравнение:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 \quad (8)$$

Главная особенность такого варианта – это простота вычислений (всего 4 коэффициента), но такой подход не дает гарантированно нулевое ускорение $\ddot{x}(t) = 2a_2 + 6a_3t$ на границах ($\ddot{x}(0) = \ddot{x}(T) = 0$).

Квинтический полином (полином 5-ой степени) имеет следующее уравнение:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 \quad (9)$$

Главное преимущество такого подхода – это нулевые ускорения на границах, но вычислительная нагрузка такого метода выше.

2. Циклоидальные траектории

Циклоидальная траектория представляется следующим уравнением:

$$x(t) = x_0 + (x_f - x_0) \left(\frac{t}{T} - \frac{1}{2\pi} \sin \left(\frac{2\pi t}{T} \right) \right). \quad (10)$$

Тогда скорость и ускорение будут:

$$\dot{x}(t) = \frac{x_f - x_0}{T} \left(1 - \cos \left(\frac{2\pi t}{T} \right) \right), \quad (11)$$

$$\ddot{x}(t) = \frac{2\pi(x_f - x_0)}{T^2} \sin \left(\frac{2\pi t}{T} \right). \quad (12)$$

Такой подход дают нулевую скорость и ускорения на границах временного отрезка $[0; T]$.

3. Библиотечные траектории на основе Motion Capture

Данный метод отличается от предыдущих так как он не основан на аналитических вычислениях, все траектории движений списываются с животных/людей и далее адаптируются под робота.

На первых этапах фиксируются маркеры на суставах (бедро, колено, стопа) в пространстве используя оптических системы типа MoCap. Далее эти данные масштабируются по длинам звеньев робота, фильтруя шумы. Этот метод позволяет роботу подражать движениям животных.

2.3 Алгоритмы управления

Метод точки нулевого момента (Zero Moment Point/ZMP):

В робототехнике термин точки нулевого момента очень часто используется в отрасли шагающих роботов. Его предложил Вукобратович в 1968 г. и этот принцип теперь лежит в основе большинства алгоритмов по передвижению.

Точка нулевого момента (ZMP) – это такая точка на опорной поверхности, где суммарный момент инерционных сил и силы тяжести не имеет составляющих вдоль горизонтальных осей. Принцип динамической устойчивости заключается в том, что если ZMP находится внутри опорного многоугольника, то робот будет сохранять равновесие.

Можно связать ZMP с положением центра масс через LIPM и получить уравнения движения маятника вдоль оси x :

$$\ddot{x}_c = \frac{g}{z_c} (x_c - x_{ZMP}). \quad (13)$$

Аналогично для оси y :

$$\ddot{y}_c = \frac{g}{z_c} (y_c - y_{ZMP}). \quad (14)$$

Напомним, что $z_c = \text{const}$. Выразим координаты ZMP через уравнения (8), (9):

$$x_{ZMP} = x_c - \frac{z_c}{g} \ddot{x}_c, \quad (15)$$

$$y_{ZMP} = y_c - \frac{z_c}{g} \ddot{y}_c, \quad (16)$$

(x_c, y_c, z_c) — это координаты центра масс робота в трехмерном пространстве.

Важно отметить, что ZMP и центр давления (CoP) — это разные понятия, хотя в динамически сбалансированной походке они совпадают. CoP — это точка, в которой можно заменить давление между стопой и землей силой.

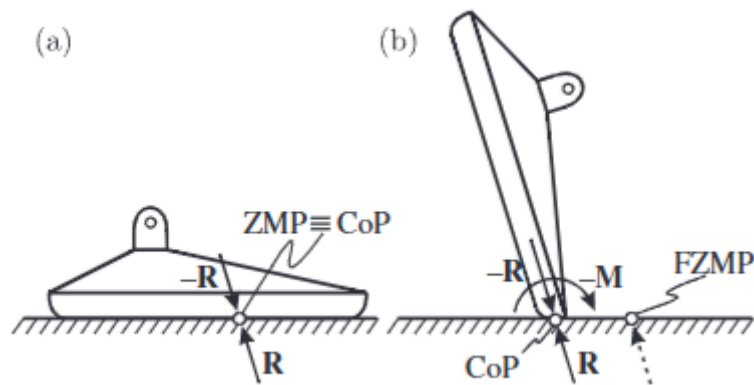


Рисунок 9 – Корреляция между ZMP и центром давления (CoP) [21]

ZMP и CoP совпадают, когда все силы и моменты уравновешены, это тот сценарий которого все хотят добиться для устойчивого движения. При наличии внешнего возмущения точка приложения силы может сместиться к краю стопы и тогда возникает фиктивный ZMP, который будет отражать величину этого момента, но будет находиться вне опорного треугольника. Такое положение сигнализирует, что мы близки к потере равновесия.

Модельно-предиктивное управление MPC

МРС — это стратегия управления, при которой на каждом шаге времени решается задача оптимального управления на конечном горизонте, используя модель системы. Затем применяется только первое управляющее воздействие, и процесс повторяется на следующем шаге.

Рассмотрим на примере упрощенной модели LPM, используя уравнения (13), (14):

$$\dot{x} = Ax + Bu,$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{g}{z_c} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{g}{z_c} & 0 \end{bmatrix}, B = \begin{bmatrix} 0 & 0 \\ -\frac{g}{z_c} & 0 \\ 0 & 0 \\ 0 & -\frac{g}{z_c} \end{bmatrix}$$

Дискретизируем систему:

$$x_{k+1} = A_d x_k + B_d u_k, \quad (18)$$

$$A_d = I + A\Delta t, B_d = B\Delta t,$$

где $x_k = [x_c, \dot{x}_c, y_c, \dot{y}_c]^T$ — вектор состояния (координаты и скорость центра масс), $u_k = [f_x, f_y]^T$ — компоненты сил, A, B — матрицы дискретизированной линейной модели.

Главная задача данного алгоритма минимизировать целевую функцию:

$$\sum_{k=0}^{N-1} \left(\|x_k - x_{ref}\|_Q^2 + \|u_k\|_R^2 \right), \quad (19)$$

где x_{ref} — желаемое состояние системы, Q, R — матрицы весов.

Общий пример управления МРС представлен на схеме ниже. Контроллер получает данные о текущем состоянии системы, включая измеренные выходные параметры и внешние возмущения, после чего использует внутреннюю модель для определения будущего поведения системы. На основе этих прогнозов вычисляется последовательность управляющих воздействий, направленных на минимизацию целевой функции в пределах заданного горизонта. В общем случае эта оптимизация позволяет снизить отклонение между предсказанными выходными параметрами системы и опорной траекторией. При этом непредсказуемые внешние возмущения оказывают влияние исключительно на поведение самой системы.

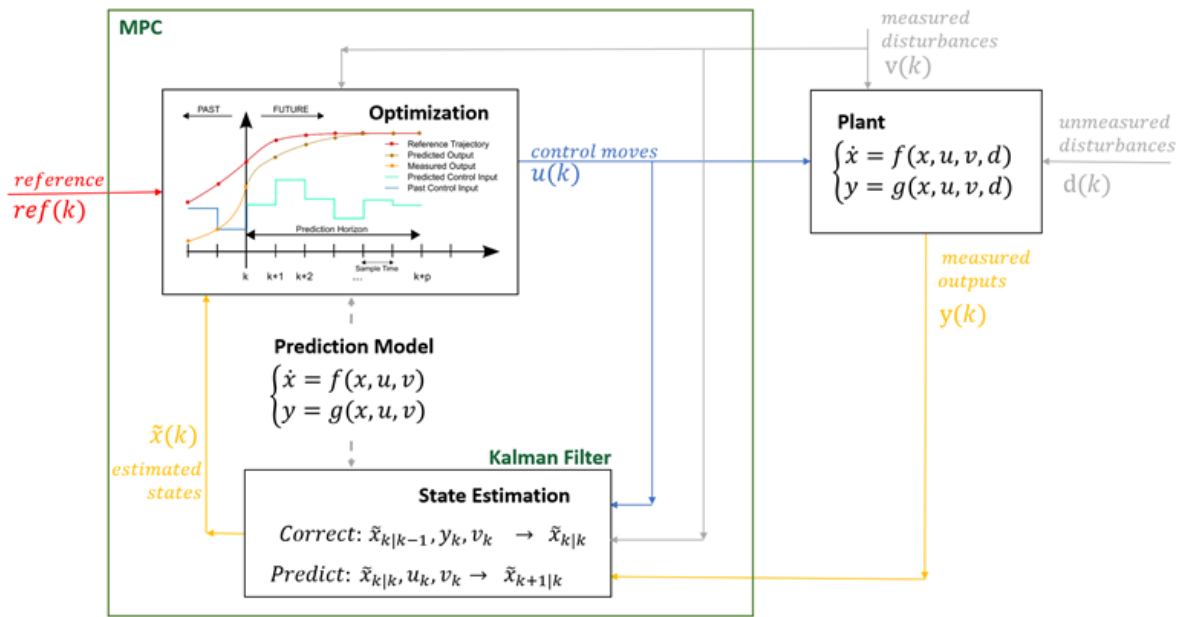


Рисунок 10 – Пример управления MPC [22]

В данной главе была рассмотрена краткая теория, стоящая за основными упрощенными моделями, алгоритмами планирования и управления движением шагающими роботами. На практике для быстроты работы компьютерных программ используются упрощенные модели из-за высокой сложности математических вычислений на каждом шаге, такие как модели линейного обратного маятника (LIPM) и пружинного обратного маятника (SLIP).

Были проанализированы базовые алгоритмы управления для поддержания устойчивости движения роботом, такие как метод нулевого момента (ZMP) и модельно-предиктивное управление (MPC). Были рассмотрены такие основные методы используемые для планировки движения, как конечный автомат, построение кривых траекторий движения ноги и виды походок четвероногих роботов.

3 ПОСТАНОВКА ЗАДАЧИ

В данной работе мы будем использовать модель робота Unitree A1 в среде MuJoCo. Если задать все углы сочленений как нули, ноги робота будут выпрямлены и расположены перпендикулярно относительно тела, однако такой случай физически нереализуем из-за ограничений конструкции данного квадрупеда. Однако эта информация помогает нам понять, как именно происходит отсчет углов поворота сочленений.

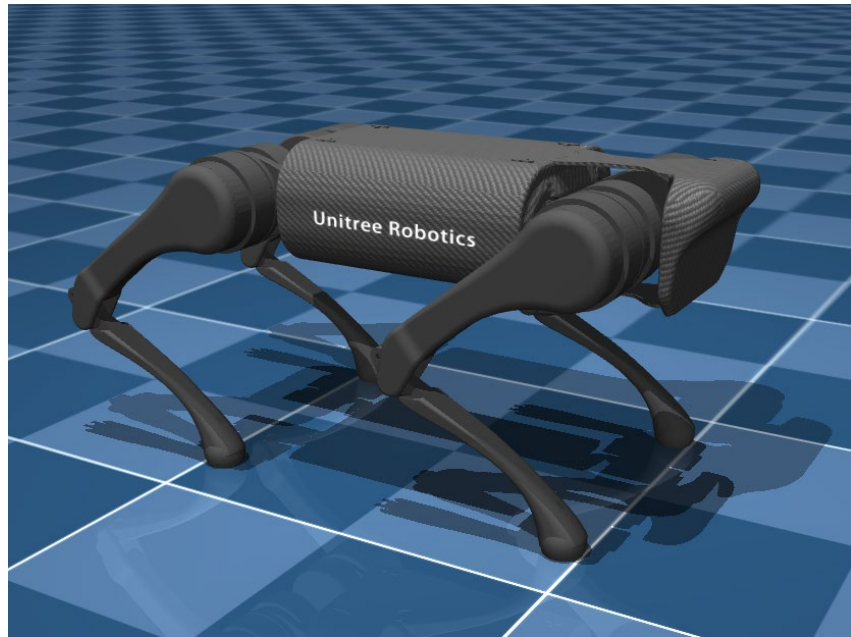


Рисунок 11 – модель Unitree A1 в MuJoCo (дефолтная позиция)

Изначальные углы в радианах одинаковы для всех четырех ног и являются таковыми:

$$\theta_1 = 0, \theta_2 = 0.9, \theta_3 = -1.8$$

Местоположение робота записываются в `data.qpos[0:7]` следующим образом:

$$[x \quad y \quad z \quad q_x \quad q_y \quad q_z \quad q_o],$$

Где x, y, z – координаты центра робота, а q_x, q_y, q_z, q_o – кватернион.

Далее в `data.qpos[7:19]` идут углы каждого сочленения каждой ноги:

$$[\theta_{FR_1} \quad \theta_{FR_2} \quad \theta_{FR_3} \quad \theta_{FL_1} \quad \theta_{FL_2} \quad \theta_{FL_3} \quad \theta_{RR_1} \quad \theta_{RR_2} \quad \theta_{RR_3} \quad \theta_{RL_1} \quad \theta_{RL_2} \quad \theta_{RL_3}]$$

Масса робота основываясь на значениях из функции `mj_getTotalmass()`:

$$m = 12.453 \text{ кг}$$

FR, FL, RR, RL – соответствуют ногам: передняя правая, передняя левая, задняя правая и задняя левая соответственно, а номера от 1-3 соответствуют сочленению тазобедренного сустава, бедра и колена.

В практической работе будет рассмотрен один сценарий: передвижение по ровной поверхности вперед.

Метрики:

- Отклонение CoM от ZMP
- Время падения робота

4 СИМУЛЯЦИЯ И РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

Для симуляции алгоритмов был использован физический движок MuJoCo (Multi-Joint dynamics with Contact) версии 3.3.0 на операционной системе Windows, AMD Ryzen 7 5800H, NVIDIA GeForce GTX 1650. Модель четвероногого робота Unitree A1 была взята из официального репозитория [23].

Порядок передвижения ног включает в себя три состояния:

- Stand: Все ноги в таком положении касаются земли (является изначальным положением робота)
- Swing: В зависимости от выбранной походки, нога находится в стадии маха по воздуху
- Stance: Нога в контакте с землей, отличается от stand тем что не все ноги касаются земли одновременно

Именно из таких состояний состоит реализованный в данной работе finite state machine (FSM). На изображении ниже представлена инициализация параметров для фаз опоры и маха.

```
def _init_swing_leg(leg_num):  
    """Инициализация параметров для ноги в swing-фазе"""  
    globals.lz_i[leg_num] = pms.lz0  
    globals.lz_f[leg_num] = pms.lz0 + pms.hcl  
    globals.lx_i[leg_num] = -0.5 * globals.xdot_ref * pms.t_step  
    globals.lx_f[leg_num] = 0.5 * globals.xdot_ref * pms.t_step  
    globals.ly_i[leg_num] = 0  
    globals.ly_f[leg_num] = 0  
  
def _init_stance_leg(leg_num):  
    """Инициализация параметров для ноги в stance-фазе"""  
    globals.lz_i[leg_num] = pms.lz0  
    globals.lz_f[leg_num] = pms.lz0  
    globals.lx_i[leg_num] = 0  
    globals.lx_f[leg_num] = 0  
    globals.ly_i[leg_num] = 0  
    globals.ly_f[leg_num] = 0
```

Рисунок 12 – Изначальные параметры для фаз stance, swing

Изначально при старте все ноги находятся в состоянии stand, после прохождения времени t_{stand} ноги переходят в состояния согласно выбранной

походке. В основном цикле идет проверка на истечение времени текущей фазы, когда она истекает нога переключается в противоположное состояние, например из *stance* → *swing*

Для реализации траектории движения ноги шагающего робота нужно определить углы в суставах, благодаря которым будет достигаться заданное положение конечности в пространстве. Это решается с помощью задачи обратной кинематики. Основываясь на данных из xml файла описывающего модель робота, то длина сустава бедра равна 0.0838, а длина бедренного звена и коленного совпадают и равна 0.2. Рассматривается три степени свободы: абдукция, тазобедренный сустав и колено, бедро и голень имеют одну длину L , а целевая точка стопы в локальной системе координат задана вектором

$$X_{ref} = [x, y, z]^T.$$

Расстояние от бедра до целевой точки:

$$l = \sqrt{x^2 + y^2 + z^2}.$$

Угол абдукции:

$$q_{abd} = \arcsin\left(\frac{y}{l}\right)$$

Угол в тазобедренном суставе:

$$q_{hip} = -\frac{1}{2}q_{knee} + \arcsin\left(\frac{-x}{l}\right)$$

Угол в колене:

$$q_{knee} = -\pi + \arccos\left(\frac{2L^2 - l^2}{2L^2}\right)$$

Для генерации плавной траектории движения шага робота используется полином 5-ой степени, так как он дает нам нулевые скорости и ускорения в начале и конце движения, что позволяет сделать шаг плавным.

Нижнеуровневое управление суставами осуществляется через ПД-регулятор и компенсацию гравитации при контакте с поверхностью. Эмпирически были подобраны коэффициенты $k_p = 100, k_d = 10$.

4.1 Симуляция

PID

В данном алгоритме будет применяться ПИ-регулятор для следования за траекторией ZMP. Изучим поведение при походке ‘trot’.

$$K_{p_x} = 10, K_{i_x} = 0.5, K_{p_y} = 100, K_{i_y} = 1$$

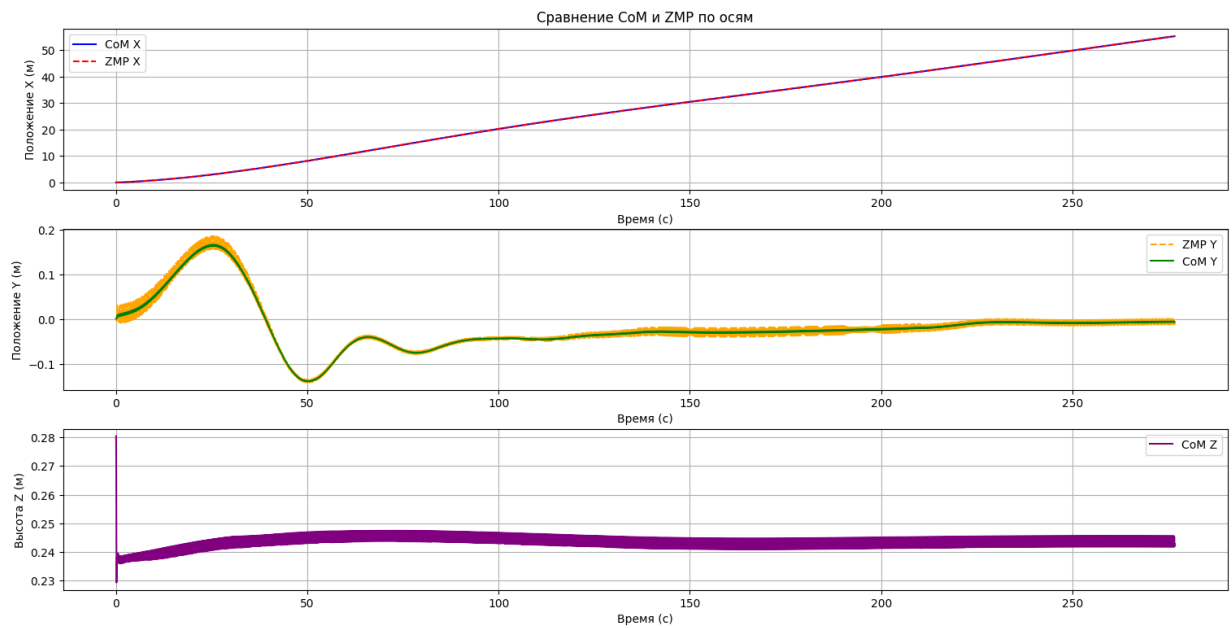


Рисунок 13 – График зависимости $CoM(t), ZMP(t)$

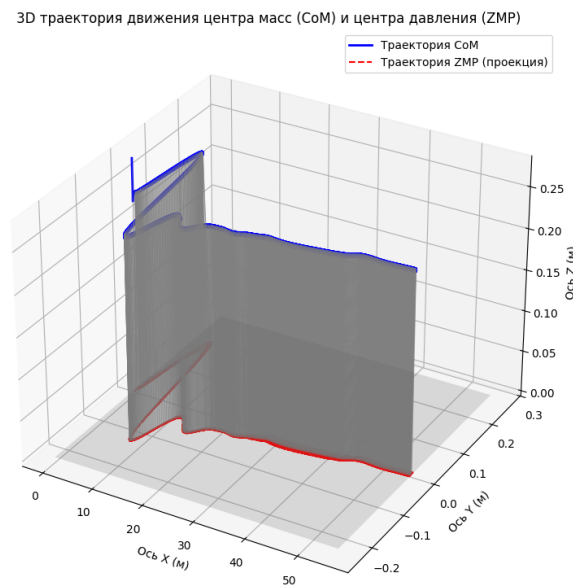


Рисунок 14 – График траектории движения CoM и ZMP в пространстве

Таблица 2 – Метрики

Время до идеальной траектории, с	228
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.013
MAE между CoM и ZMP по Y, м	0.003

$$K_{p_x} = 20, K_{i_x} = 1, K_{p_y} = 200, K_{i_y} = 5$$

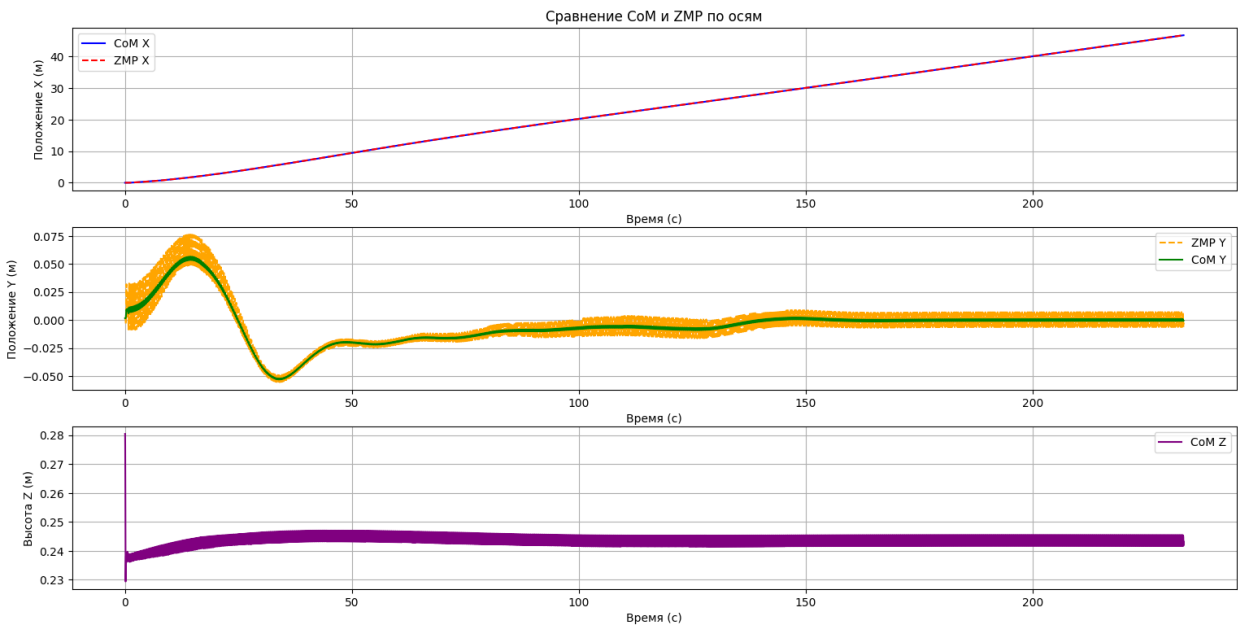


Рисунок 15 – График зависимости $CoM(t), ZMP(t)$

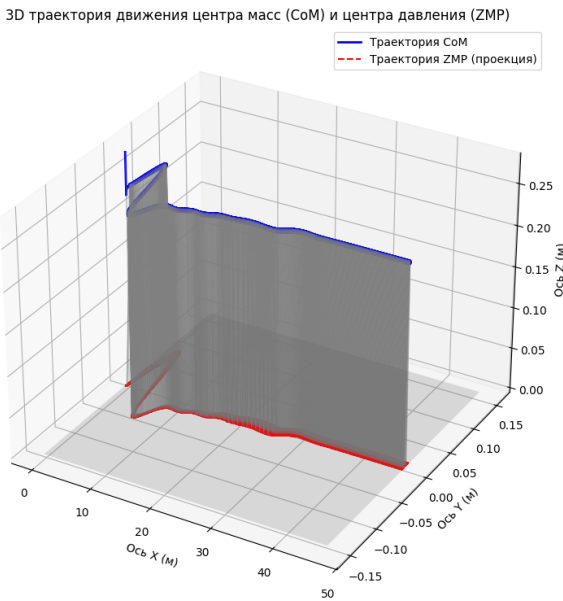


Рисунок 16 – График траектории движения CoM и ZMP в пространстве

Таблица 3 – Метрики

Время до идеальной траектории, с	112
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.013
MAE между CoM и ZMP по Y, м	0.002

$$K_{p_x} = 20, K_{i_x} = 0.5, K_{p_y} = 200, K_{i_y} = 20$$

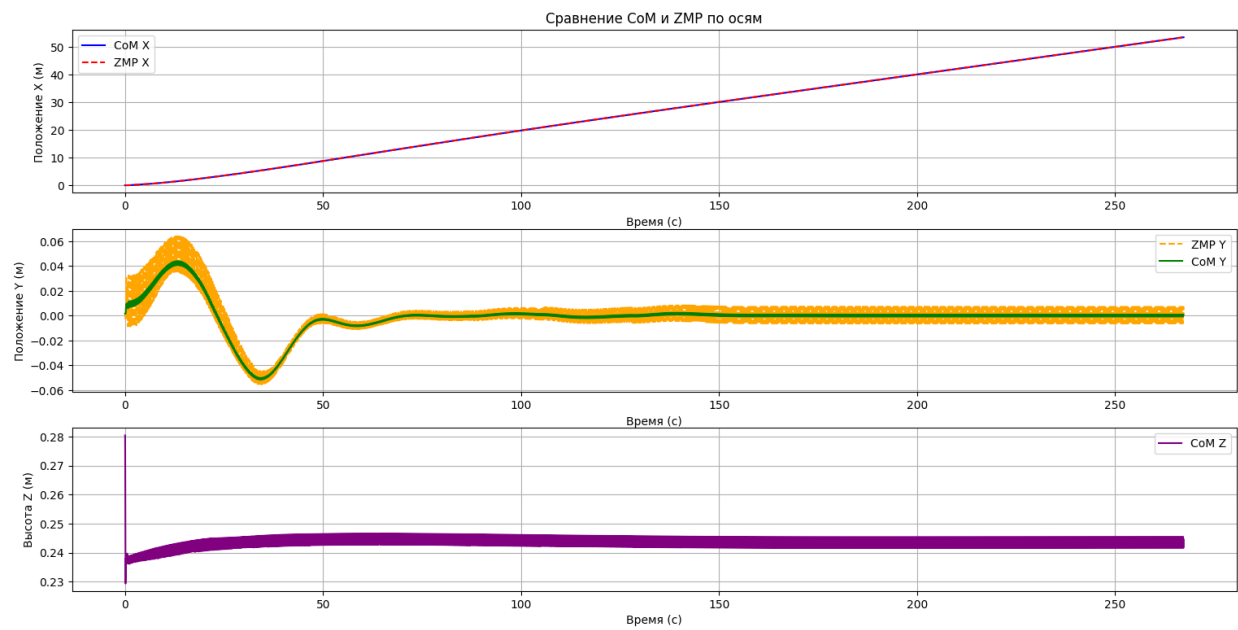


Рисунок 17 – График зависимости $CoM(t), ZMP(t)$

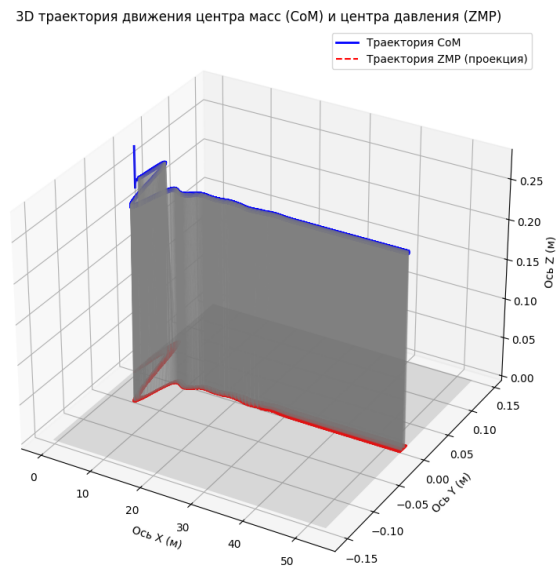


Рисунок 18 – График траектории движения CoM и ZMP в пространстве

Таблица 4 – Метрики

Время до идеальной траектории, с	66
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.013
MAE между CoM и ZMP по Y, м	0.002

MPC

$$Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

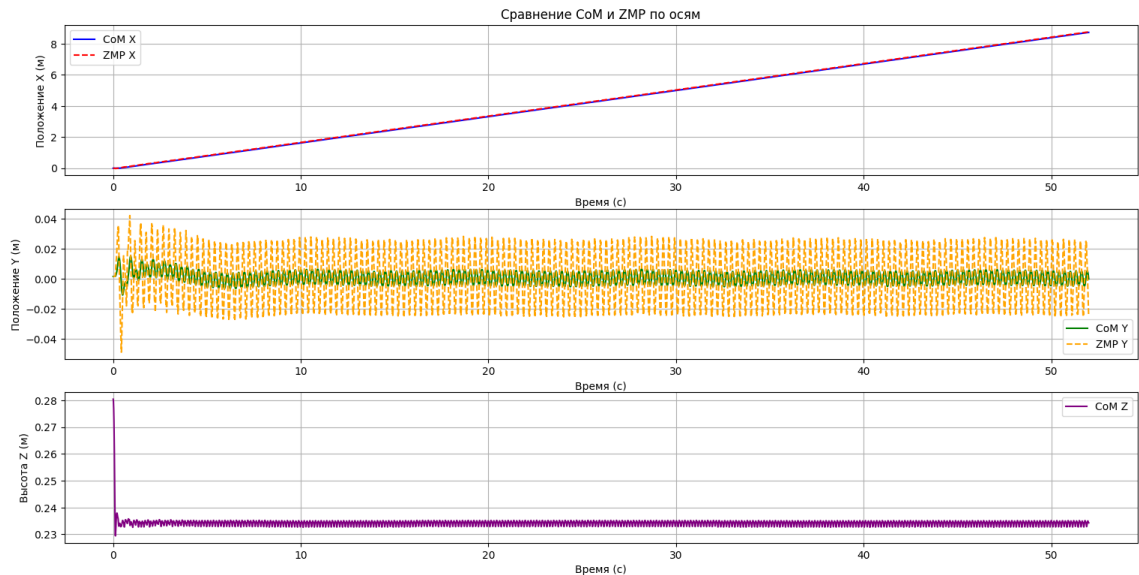


Рисунок 19 – График зависимости $CoM(t), ZMP(t)$

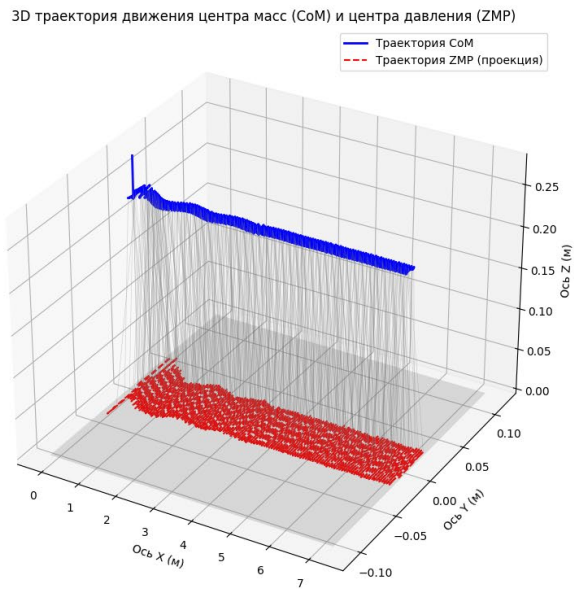


Рисунок 20 – График траектории движения CoM и ZMP в пространстве

Таблица 5 – Метрики

Время до идеальной траектории, с	5
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.043
MAE между CoM и ZMP по Y, м	0.015

$$Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 30 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

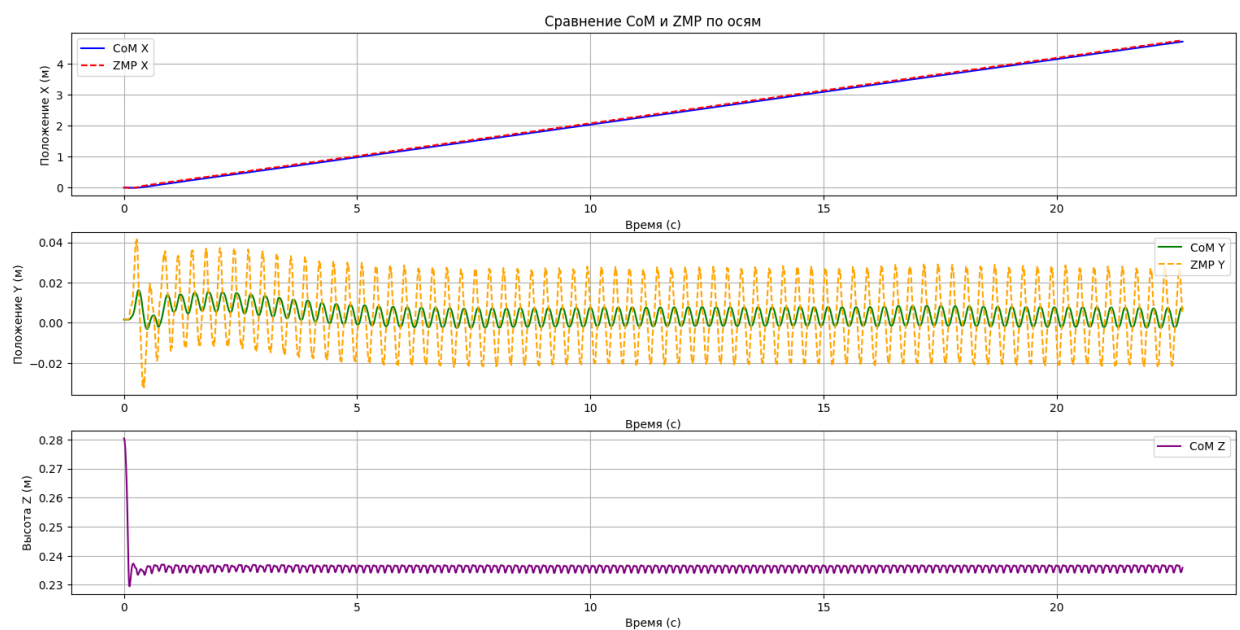


Рисунок 21 – График зависимости $CoM(t), ZMP(t)$

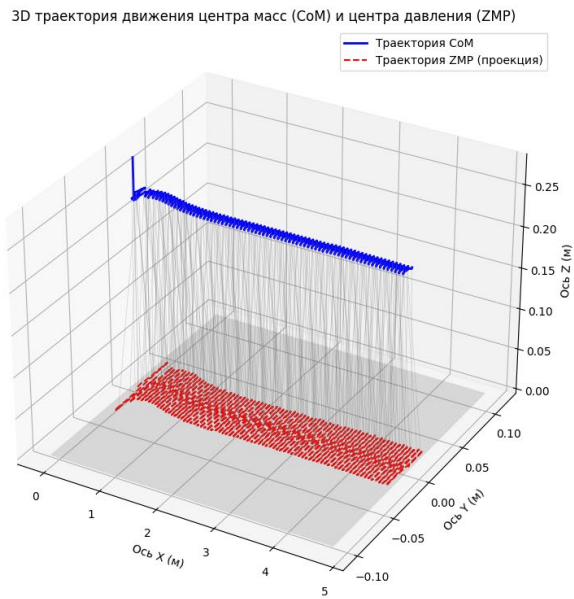


Рисунок 22 – График траектории движения CoM и ZMP в пространстве
Таблица 6 – Метрики

Время до идеальной траектории, с	7
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.042
MAE между CoM и ZMP по Y, м	0.013

$$Q = \begin{bmatrix} 30 & 0 & 0 & 0 \\ 0 & 40 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

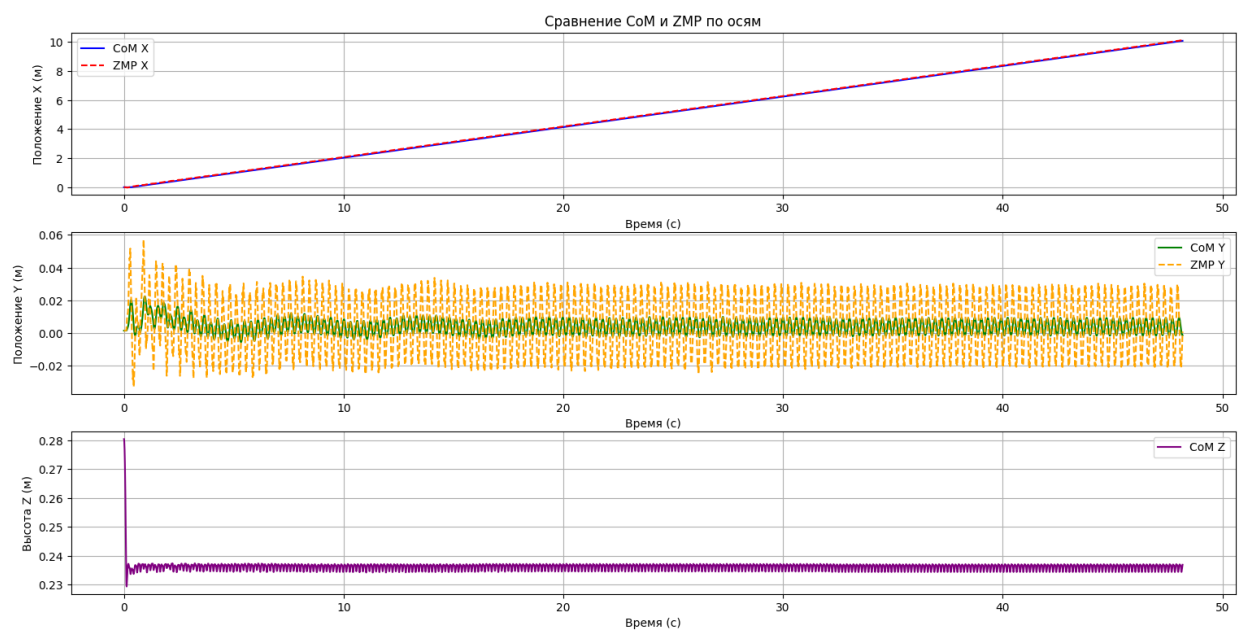


Рисунок 23 – График зависимости $CoM(t), ZMP(t)$

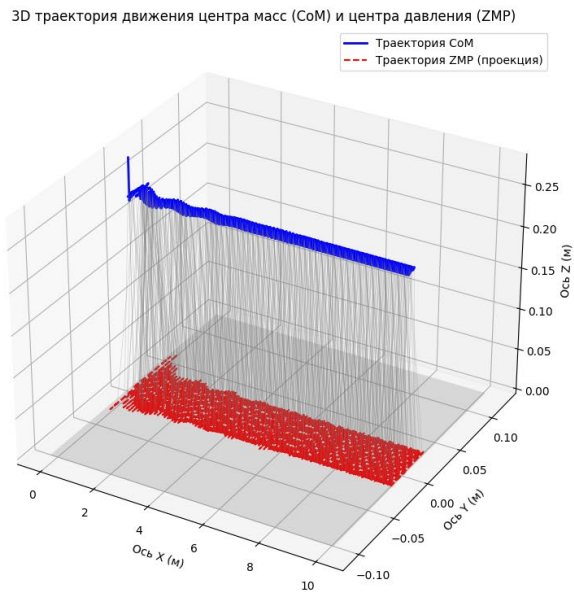


Рисунок 24 – График траектории движения CoM и ZMP в пространстве

Таблица 7 – Метрики

Время до идеальной траектории, с	7
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.043
MAE между CoM и ZMP по Y, м	0.015

$$Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R = \begin{bmatrix} 0.5 & 0 \\ 0 & 2 \end{bmatrix}$$

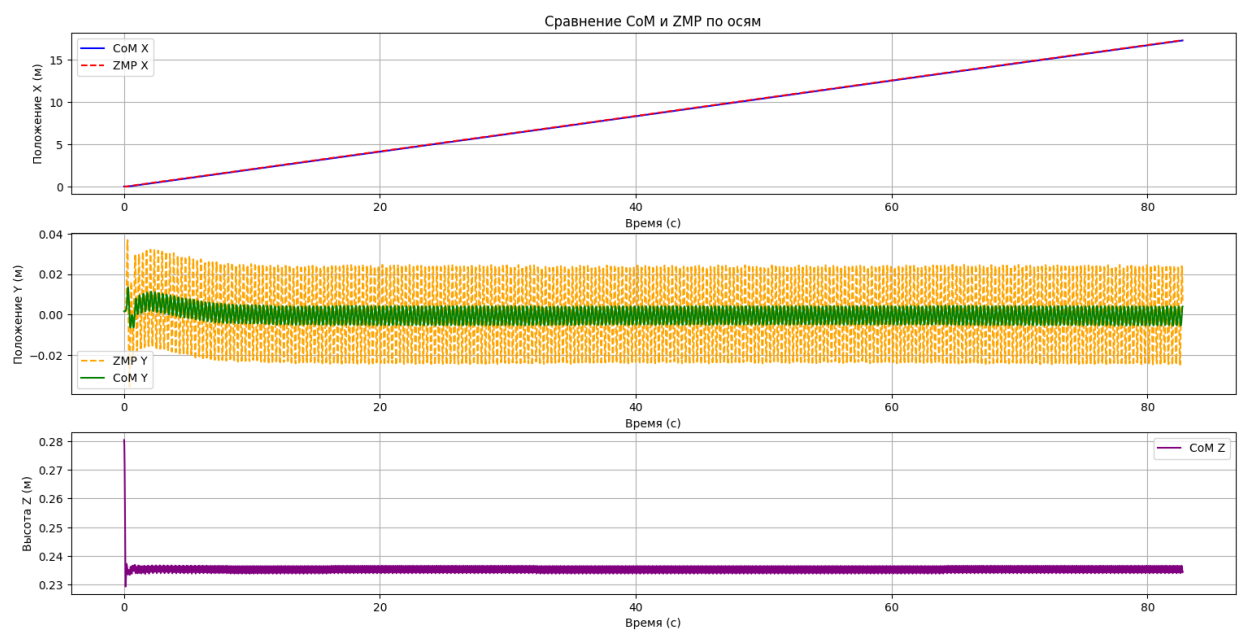


Рисунок 25 – График зависимости $CoM(t), ZMP(t)$

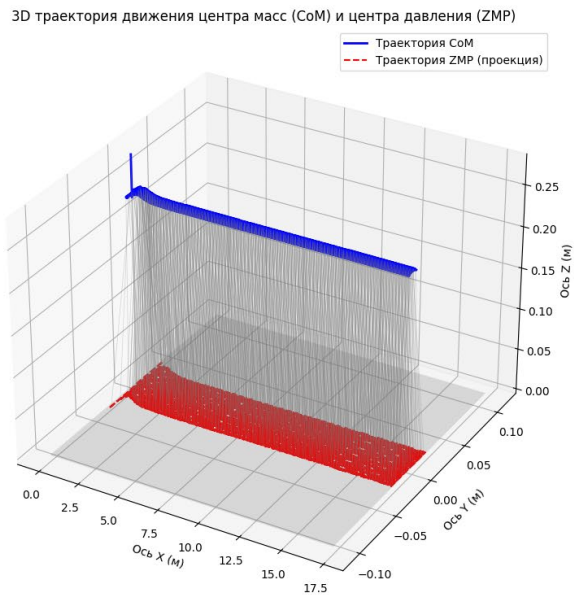


Рисунок 26 – График траектории движения CoM и ZMP в пространстве

Таблица 8 – Метрики

Время до идеальной траектории, с	6.7
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.042
MAE между CoM и ZMP по Y, м	0.012

$$Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}$$

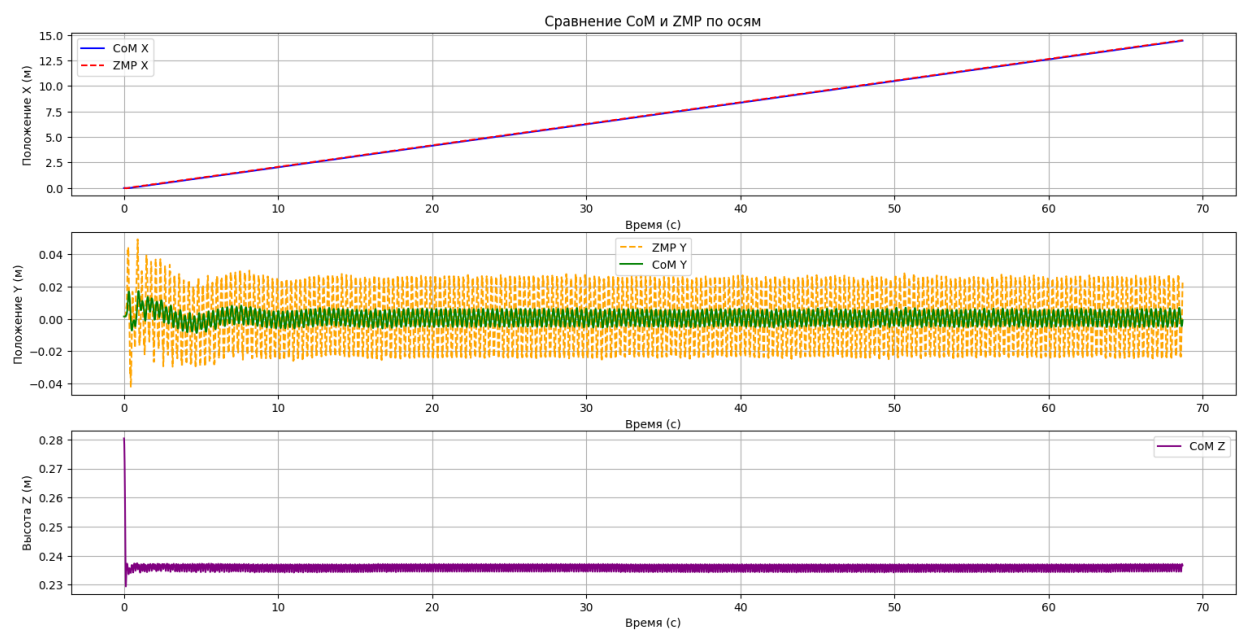


Рисунок 27 – График зависимости $CoM(t)$, $ZMP(t)$

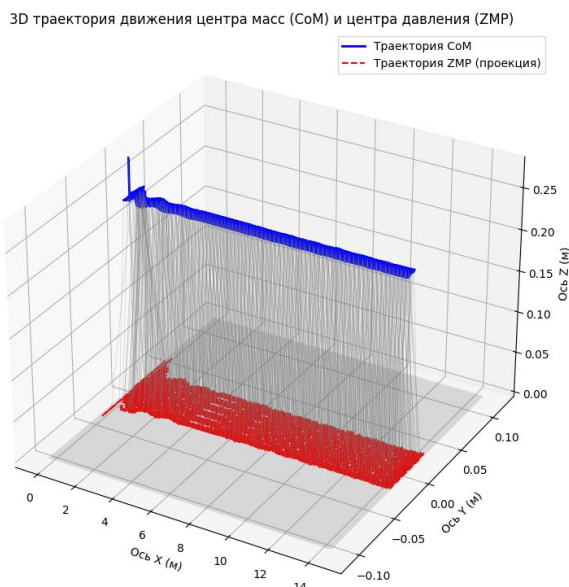


Рисунок 28 – График траектории движения CoM и ZMP в пространстве

Таблица 9 – Метрики

Время до идеальной траектории, с	6.7
Амплитуда колебаний CoM по Z, м	0.051
MAE между CoM и ZMP по X, м	0.043
MAE между CoM и ZMP по Y, м	0.014

4.2 Анализ результатов экспериментов

На основе экспериментов в рамках эксперимента движения вперед походкой “trot” со скоростью 0.2 м/с оба алгоритма показали себя стабильно, однако явно выигрывает алгоритм с модельно-предиктивным управлением, была использована упрощенная модель 3хмерного обратного маятника. Амплитуда колебаний центра масс по оси Z оставалась стабильной и одинаковой при всех тестах. Основываясь на скорости достижения нужной траектории MPC явно опережает PID, но увеличение интегральной составляющей существенно сокращает время стабилизации. А если увеличивать вес компоненты по Y в Q ошибка снизится. Более высокие ошибки у MPC связаны с использованием упрощенной модели, которая была выбрана чтобы сократить вычисления, так как MPC является куда более медленным алгоритмом чем PID, на основе экспериментов было выявлено, что скорость симуляции PID 1.04x real-time, в то время как MPC 0.22x real-time.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного исследования были рассмотрены базовые алгоритмы планирования и управления шагающими роботами, такие как PID-ZMP, MPC, DRL. Был проведен сравнительный анализ алгоритмов управления, представленный в Таблице 1. С использованием модели Unitree A1 в среде MuJoCo были реализованы алгоритмы PID-ZMP и MPC при походке trot, позволяющее стабильно ходить прямо по оси X. Экспериментально были подтверждены теоретические сведения о преимуществах и ограничениях каждого подхода.

Для реализации работы роботы была сделана система управления на основе конечного автомата с состояниями стойки, маха и опоры ног, также для планирования движения ног был использован полином 5ой степени. Разработаны два принципиально различных алгоритма: PID показал высокую точность при сопровождении траектории, но также явно указал на зависимость от точной настройки коэффициентов, MPC на основе трехмерного маятника проявил себя лучше в плане скорости выхода на целевую траекторию заметно обходя PID регулятор.

Преимуществом MPC будет тот факт, что он в 10-30 раз быстрее достигает целевой траектории, имеет лучшую адаптивность к изменяющимся условиям в теории, а прогнозирующий характер позволяет на ограниченном промежутке предугадать действия. Однако PID побеждает в малой вычислительной требовательности, опережая реальное время на 4%, также является куда более простым в реализации.

Разработанная система может служить базой для создания более сложных алгоритмов. Экспериментально была доказана эффективность MPC для быстрой стабилизации движения. В дальнейших исследованиях планируется добавление других походок, замена модели обратного маятника на более точную и реализация гибридного управления посредством MPC+WBC, а также хождение с наличием препятствий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Boston Dynamics. Atlas | Partners in Parkour [Видео] // YouTube. – 2021. – URL: https://www.youtube.com/watch?v=-e1_QhJ1EhQ
2. Hwangbo J. et al. Learning agile and dynamic motor skills for legged robots //Science Robotics. – 2019. – Т. 4. – №. 26. – С. eaau5872. Ha S. et al. Learning to walk in the real world with minimal human effort //arXiv preprint arXiv:2002.08550. – 2020.
3. Patrick Wensing, Michael Posa, Yue Hu, Adrien Escande, Nicolas Mansard, et al.. Optimization Based Control for Dynamic Legged Robots. IEEE Transactions on Robotics, 2024, 40, pp.43-63.
4. Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and Model Predictive Control
5. Peng X. B. et al. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills //ACM Transactions On Graphics (TOG). – 2018. – Т. 37. – №. 4. – С. 1-14.
6. Sylvain Koos, Jean-Baptiste Mouret, and Stéphane Doncieux. 2010. Crossing the reality gap in evolutionary robotics by promoting transferable controllers. In Proceedings of the 12th annual conference on Genetic and evolutionary computation (GECCO '10). Association for Computing Machinery, New York, NY, USA, 119–126. <https://doi.org/10.1145/1830483.1830505>
7. Buchli, J., Stulp, F., Theodorou, E., & Schaal, S. (2009). Model predictive control for legged robots. В Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), St. Louis, MI, USA, 11–15 октября 2009, стр. 1–8.
8. Camacho, E.F., & Bordons, C. (2007). Model Predictive Control. Berlin/Heidelberg, Germany: Springer Science & Business Media.
9. Di Carlo, J., Katz, B., Bledt, G., & Kim, S. (2018). Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control. В Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Мадрид, Испания, 1–5 октября 2018, стр. 1–9.
10. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. arXiv, arXiv:1801.01290.
11. Ishihara, K., Itoh, T.D., & Morimoto, J. (2020). Full-Body Optimal Control Toward Versatile and Agile Behaviors in a Humanoid Robot. IEEE Robot. Autom. Lett., 5, 119–126.
12. Chen M. et al. Analysis and optimization of interpolation points for quadruped robots joint trajectory //Complexity. – 2020. – Т. 2020. – №. 1. – С. 3507679.

13. Thomas, F., Santamaria-Navarro, A., & Andrade-Cetto, J. (2014). Toward lifelong adaptive mobile robots. В Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Чикаго, IL, США, 14–18 сентября 2014, стр. 1585–1591.
14. Winkler, A.W., Bellicoso, D.C., Hutter, M., & Buchli, J. (2018). Gait and trajectory optimization for legged systems through phase-based end-effector parameterization. IEEE Robot. Autom. Lett., 3, 1560–1567
15. Haarnoja T., Ha S., Zhou A., Tan J., Tucker G., Levine S. Learning to Walk via Deep Reinforcement Learning // arXiv preprint arXiv:1812.11103. – 2019. – URL: <https://arxiv.org/abs/1812.11103>
16. Caron S., Point mass model [Электронный ресурс] // *scaron.info*. – URL: <https://scaron.info/robotics/point-mass-model.html> (дата обращения: 23.04.2025).
17. Kajita S., Kanehiro F., Kaneko K., Yokoi K., Hirukawa H. The 3D linear inverted pendulum model: A simple modeling for a biped walking pattern generation // *Proceedings of the 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Maui, HI, USA, Oct. 29 – Nov. 3, 2001. – P. 239–246.
18. Fan, Y., Pei, Z., Wang, C., Li, M., Tang, Z. and Liu, Q. (2024), A Review of Quadruped Robots: Structure, Control, and Autonomous Motion. Adv. Intell. Syst., 6: 2300783. <https://doi.org/10.1002/aisy.202300783>
19. Vukobratović M., Borovac B. Zero-moment point — thirty five years of its life // International journal of humanoid robotics. – 2004. – Т. 1. – №. 01. – С. 157-173.
20. MathWorks. What Is Model Predictive Control? [Электронный ресурс]. – 2023. – Режим доступа: <https://www.mathworks.com/help/mpc/gs/what-is-mpc.html> (дата обращения: 29.04.2025).
21. Документация MuJoCo [Электронный ресурс]. – Режим доступа: <https://mujoco.readthedocs.io/en/stable/models.html> (дата обращения: 12.05.2025)
22. Изображение [Электронный ресурс]. – Режим доступа: https://www.researchgate.net/publication/308842717_Enlarging_the_region_of_stability_using_the_torque-enhanced_active_SLIP_model/figures?lo=1 (дата обращения: 12.05.2025).
23. Kang D., De Vincenti F., Adami N. C., Coros S. Animal Motions on Legged Robots Using Nonlinear Model Predictive Control // 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). – IEEE, 2022. – С. 11955-11962. – DOI: 10.1109/IROS47612.2022.9981945.